

Necesidad del orquestador

Objeto.

El presente documento pretende presentar el problema existente en la gestión de las máquinas virtuales en nuestra universidad. Asimismo, se presentan las necesidades que se presentan y las posibles soluciones que una aplicación de orquestación solucionaría.

Antecedentes

En la escuela de informática se imparten diferentes asignaturas en las que se ofrecen a los alumnos unos conjuntos de máquinas virtuales (VM), temporales para la realización de sus prácticas.

Cada asignatura tiene una serie de VM que se replican para cada alumno.

El profesor de la asignatura define las VM y le solicita a un administrador que genere tantos conjuntos de VM como alumnos. Esto se hace actualmente con unos hipervisores VMWare. Se crean unas VM base que actúan como plantillas. Las VM que se generan para los alumnos son Linked clones de estas. Al alumno se le da la IP de la VM y lo único que puede hacer es entrar en ella y actuar desde dentro del sistema operativo que tiene dentro. Si el alumno para la VM, saliendo del OS, es el administrador el que tiene que arrancarla de nuevo. En casos de corromper el sistema, cosa que los alumnos pueden hacer accidentalmente, el administrador tiene que eliminar y recrear la VM desde la imagen base.

Las máquinas se generan con ayuda de unos scripts que leen una definición de VM, y una lista de alumnos. Las IPs se gestionan de forma semi manual. Existe un DHCP en la red donde están las VM, de forma que hay una serie de IPs estáticas en el servidor DHCP. El truco es que las MAC de las VM tienen codificada en su numeración la IP deseada. Las asignaciones del DHCP saben cuáles son estas traducciones y asignan correctamente las IP.

Problemas

Aparte de los problemas que ya se intuyen, existe la casuística de la aparición de un nuevo alumno cuando se ha desplegado ya el conjunto de las VM de la asignatura.

Uno de los principales problemas es la gestión de tantas VM. Si en una asignatura hay 30 alumnos y cada uno va a tener 15 VM, son muchas máquinas que desplegar.

Los scripts actuales están hechos por el administrador, para interactuar con los hipervisores existentes de VMWare. Si en el futuro se cambia de hipervisores, se necesitarán otros, adaptados a la API de esos.

Si multiplicamos el número de VM por el número de asignaturas, vemos que el administrador tiene mucha carga de trabajo para despliegue, actualizaciones y recreación de VM.

Asimismo, ahora, un alumno que entre en otra IP, puede ver las máquinas de otro compañero. Aunque esto es muy raro que ocurra.

Necesidades

Se busca un sistema que resuelva varios problemas.

Que el alumno pueda hacer snapshots y rollbacks de sus VM, esto evitaría la intervención del administrador.

Que el alumno tenga acceso a ciertas funciones del hipervisor. En principio: parar, pausar, arrancar VM. Incluso, eliminar la VM y recrearla desde la plantilla base asignada. Esto permite que las VM no estén creadas realmente si no se necesitan, el alumno la crea cuando la necesita. Lógicamente, el alumno no puede crear lo que quiera, sólo las máquinas asignadas por el profesor. Es decir, tiene una lista de VM posibles, y cada una esté prefijada cómo será. El alumno sólo puede crear lo que se le ha predefinido.

Los alumnos y profesores no tienen porqué entender de hipervidores. Lo ideal sería un interface web, donde apecerza fácilmente y de forma intuitiva las posibilidades de lo que pueden hacer.

Idealmente, el profesor y alumno interactúan únicamente con el orquestador para las operaciones permitidas. El orquestador es el que se comunica con los hipervisores, y le dice el estado de las máquinas al usuario. Así, el usuario puede ver qué máquinas están activas y sus IP.

De esta forma el usuario se abstrae de los detalles del hipervisor.

Un sistema así, además podría manejar varios hipervisores. Es más, los hipervisores podrían ser de diferentes tipos sin que el usuario sepa de eso. Hay que tener en cuenta que el usuario sólo utilizará un subconjunto de operaciones de manejo de VM que todos los hipervisores tienen: clonar, arrancar, parar, pausar, hacer snapshots, borrar...

Así, el administrador determina donde estará el tipo de VM y el usuario solo sabe que puede hacer ciertas operaciones, sin preocuparse de almacenamiento ni detalles.

La idea es que el administrador, defina una VM plantilla. Estas tienen los detalles de en qué hipervisor estarán, así como los parámetros de hardware virtual, incluso el OS instalado. Siempre se basarán en una VM base preconfigurada. Los usuarios no van a instalar OS en las VM.

Solución

A este nivel de diseño, lo que entendemos como solución es la arquitectura software que define la aplicación del orquestador. No entremos en la implementación. Como comentario, la implementación seguramente se haga con un servicio codificado en python, dada la facilidad de librerías y frameworks de python que permiten hacer de todo. Como será en linux, python es ideal para esto.

Se ha pensado una posible solución de arquitectura, pero el sistema está abierto a otras ideas.

Algo que tiene bastante sentido es que la solución sea más general, no sólo enfocada a universidad/asignaturas/alumnos/VM. Se ve claramente que se podría generalizar a empresa/departamento/VM. o a organización/centro de investigación/ laboratorio/ departamento/ VM. O cualquier jerarquía.

Si el orquestador permite diferentes estructuras, será más universal.

La solución que se decida, es importante que, desde el punto de vista del usuario, sea útil, versátil, y entendible. Para un usuario no experto, parece lógico que sea una metáfora de algo conocido o que requiera un mínimo salto mental.

Solución: árbol de descriptores

La actual solución es, por una parte, el concepto mental que se quiere instalar en el usuario. Es importante que la estructura mental en el usuario sea clara y fácil de entender, a la vez que útil y potente para lo que se busca.

Se ha pensado en un concepto: el descriptor. El descriptor podría ser una analogía de un descriptor de un fichero, que internamente apunta a un sistema de ficheros, que puede estar en cualquier sistema de ficheros, y en cualquier soporte, pero eso lo sabe el OS, el usuario hace operaciones estándar en el

descriptor del fichero, y se ejecutan. Es más, el usuario no sabe de descriptor, para él, el descriptor es el propio fichero.

Similarmente, el orquestador ofrece descriptor de VM, aunque para el usuario eso simboliza la propia VM.

El descriptor, por tanto, es más útil para describir la arquitectura. Este objeto mantiene información diversa en si mismo: apunta al hipervisor, y a la VM en el. Además, tiene las credenciales de acceso al hipervisor, y sabe de que tipo es. Vale como interface para ejecutar las operaciones típicas en el hipervisor sobre su VM. Además incorpora un estado interesante: la posibilidad de una máquina provisionada. Es decir, en estado provisionado, la máquina no está realmente desplegada en el hipervisor. Sólo se guarda la información para su creación, pero el hipervisor no sabe nada. Es una forma de guardar un hueco, una máquina fantasma que no existe, pero que el usuario puede instanciar, desplegar. De esta forma el usuario verá todas las máquinas que le han sido asignadas, aunque estén sólo provisionadas. Podrá desprovisionarlas y recrearlas, es decir desplegarlas de nuevo.

Este concepto unifica en un solo objeto, la definición de una VM, y la VM en si misma. Este descriptor es el unico interface, por eso el usuario lo verá como la VM, con posibilidad de un estado nuevo que los hipervisores no contemplan: estado provisionado.

¿quien crea los descriptors? La idea es que sea el profesor de la asignatura, como usuario más avanzado, el que, cree, en un espacio asignado a la asignatura, las VM de los alumnos. Más bien dicho, los descriptors. Cada alumno podrá desplegarlas.

Para facilitar la tarea, existirá una aplicación sobre el orquestador, o una extensión si se quiere, que automatiza esto. El profesor define un conjunto de VM genericas para la asignatura, y la aplicación replica esto para cada alumno. Es más, se encarga de actualizar los descriptors según se cambie la lista de alumnos.

El árbol: se propone una estructura de arbol para guardar los descriptors. Esto permite organizar estructuras de asignaturas/alumnos/VM o cualquier otra. El arbol es muy flexible y potente.

Otra ventaja de la estructura de árbol es que puede permitir las definiciones jerárquicas. Es decir, si hay varias VM que comparten definición, se pueden heredar propiedades de conjuntos de definiciones comunes en una carpeta. Por ejemplo, si varias VM se basan en la misma plantilla, en lugar de definir esa plantilla en cada descriptor, todos los descriptors que están en una carpeta, pueden heredar las propiedades comunes de una carpeta. Esa, a su vez, pueden heredar de su carpeta padre. Así, se pueden tener características comunes a cada asignatura, y cada alumno. Por ejemplo, toda una asignatura tiene el mismo manager, el profesor, y todas las VM de un alumno comparten al usuario alumno concreto, como usuario final de esas máquinas. Las plantillas de VM permitidas son también comunes para cada asignatura. Este sistema de propiedades heredadas puede servir también para definir los roles y permisos.

Esta jerarquia permite que un profesor defina sus máquinas basándose en unos nombres de plantillas que el administrador le deja, sin que sepa los detalles de su definición. El administrador podría cambiar la definición de las máquinas base y el profesor no tener que cambiar las definiciones de sus VM de la asignatura.

Esas son algunas de las ventajas de estas definiciones en cascada. La idea es que, cada carpeta tenga una configuración. Así, cada VM tiene una definición local y una definición completa heredada. La definición completa es recursiva: la de su carpeta, sobre la que se une la local. De esta forma, hay una definición asociada al raíz, que se va completando en cascada. En cualquier punto se pueden completar y sobrescribir propiedades a la definición.

Algunas de las propiedades que se definen:

- Definiciones de hipervisores: nombre alias, tipo, IP, credenciales...
- Definición de VM plantilla: nombre de VM base y en qué hipervisor está, atributos hardware que sobrescriban a la base al hacer un clon. Ubicación de almacén donde se guardará el clon.
- Definición de una plantilla, basada en otra plantilla,
- Definición de permisos, roles, asignación de usuarios a roles

Respeto a los permisos se ha pensado en una base de datos de usuarios, que en principio podría ser una lista interna o externa de usuarios-contraseñas. El tema de la autenticación será algo estándar, no se ha pensado mucho en ello. Como habrá seguramente una API REST y interface web, se dejará a los framework que usen las tecnologías estándar de autenticación en http.

Se definen 3 roles:

- superusuario: el administrador
- manager: el profesor o similar
- end_user: el alumno o similar

El manager tendrá acceso a unas carpetas concretas que el superusuario le da. Podrá crear carpetas de ahí hacia abajo. Podrá crear descriptors de máquinas basadas en las máquinas que le sean permitidas. Podrá crear plantillas a partir de las plantillas base que le sean asignadas. Podrá usar la herramienta de ayuda de despliegue de asignaturas. U otras que se definan en el futuro. Son ayudas que trabajan sobre el orquestador.

Los usuarios solo podrán actuar sobre las máquinas(descriptors) asignados, No podrán hacer mucho más.

A cada nivel del árbol se pueden asignar usuarios a cada rol. Esta asignación también necesita permisos. Un manager puede asignar alumnos y otros manager a sus recursos.

En principio, cada rol tiene unos permisos definidos, pueden hacer ciertas operaciones prefijadas. En el futuro se podrán personalizar algo más por carpetas.

En un futuro se podrá establecer un sistema de cuotas. No importante de momento

Otra especificación que se ha pensado, es que el orquestador no afecte a los hipervisores existentes. Esto quiere decir que los hipervisores seguirán funcionando como siempre, sin efectos laterales. En caso de caída del orquestador, o simplemente no querer usarlo, se podrá usar normalmente como siempre con sus herramientas. Es más, para la solución actual, la creación de las VM base, se hace directamente con el hipervisor correspondiente.

Esto hace que la instalación del orquestador en una infraestructura no sea invasiva ni tenga efectos laterales. Lo normal es que, si se consigue que funcione bien, la mayoría del trabajo diario se haga con el orquestador.

La solución propuesta es ampliable. Se trata de una capa que deberá funcionar muy estable. Sobre este orquestador se podrán añadir aplicaciones, como la indicada de despliegue de asignaturas, o otras de auditoría o seguridad que se decidan.

Como se ve , se trata de hacerlo lo mas universal posible, en el sentido de que valga para hipervisores actuales y para futuros, añadiendo los conectores correspondientes para la comunicación con los hipervisores de los tipos que se necesiten.

Arquitectura

Para esta solución se ha pensado en los siguientes elementos:

El orquestador será un servicio linux, corriendo en un host.

Ofrecerá una API REST para su manejo desde clientes. En principio un cliente remoto CLI, de llamadas desde la linea de comando, desde un CLI interactivo y opciones de lanzarlo con scripts.

El mismo servicio ofrecerá un interface Web

El orquestador tiene un core que se encarga de ejecutar las operaciones. Habrá operaciones síncronas y asíncronas. Las operaciones sobre hipervisores, se realizan mediante unos conectores.

Conectores a hipervisores: Módulos que tiene el orquestador para comunicarse con los hipervisores. Internamente, todas las operaciones se hacen a un interface común. Se establece un protocolo de operaciones comunes. Las llamadas son enrutadas a cada conector, según el tipo de hipervisor de corresponde a cada descriptor. Cada conector es un cliente de la API del tipo de hipervisor. Por ejemplo, para VMWare se usa la librería python que VMWare ofrece. Para Proxmox, es un cliente REST que también existe, etc.

Almacenamiento. El servicio mantendrá la estructura del árbol y demás configuración en memoria y disco, de forma que es permanente. Al arrancar el servicio, deberá mantener la coherencia con el estado real de las VM en los hipervisores.

Descriptores. Como se ha comentado son objetos que guardan toda la configuración, así como la conexión con la VM real. Pueden estar en estado Provisionado o desplegado, según la VM está creada o no. También en estado desconectado, si se pierde temporalmente la conexión con el hipervisor. Saben cómo compilar la definición completa, trayendo recursivamente la de su carpeta padre y combinando con la configuración local.

Árbol. Es la estructura que almacena carpetas y descriptores. En cada carpeta podrá haber ficheros especiales, como la definición común para esa carpeta. Y puede que otros relacionados con los temas de permisos. U otros que aplicaciones futuras puedan requerir. Su implementación interna es invisible para los usuarios. En una primera aproximación podría ser una carpeta en el propio servidor que ejecuta el servicio. O una base de datos local.

Hipervisores. No forman parte del orquestador en sí. Han de existir. No se necesita que estén en el mismo host. El orquestador tendrá definido, en las definiciones guardadas en los descriptores y/o definiciones comunes en carpetas, los enlace a los hipervisores. Cada definición de hipervisor tendrá su dirección y credenciales y un alias interno. De esta forma, cuando se define una VM o plantilla, esta referencia al alias interno que apunta al hipervisor. Asimismo, se guarda en cada descriptor, la referencia a la VM que cada hipervisor conoce, su nombre en el hipervisor. Puede interesante también, que en un campo de metadatos libre en el VM en el hipervisor, se guarde alguna información de referencia inversa, con datos del descriptor.

Coherencia. Para mantener la coherencia, las VM reales creadas son las que mantienen la verdad. Cuando se consulte el estado de una VM, será la información del hipervisor, no parece necesario guardar una caché de esos datos. La definición de las VM está en el orquestador, pero podría pasar que se cambiara algo de una máquina funcionando. Desde el propio orquestador esto se prohíbe, no se puede cambiar definiciones de una VM desplegada, para evitar inconsistencias. Habrá que tener cuidado con las definiciones en cascada, eso supone un cuidado especial de qué hacer si se cambia algo que se propaga. Habrá que estudiar las opciones con cuidado.

Si se cambia algo directamente en el hipervisor, la VM está en modo inconsistente. La lógica será dejarlo así hasta que se desprovisione.

Futuras especificaciones

Se pretende que en un futuro, se pueda desplegar el orquestador en un sistema de alta disponibilidad (HA), con lo que se tendrá que refactorizar para poder instalarse en un cluster o en 2 máquinas con algún sistema de sincronización. Ya existen soluciones buenas para mantener 2 servicios así. Lo que bará que tener en cuenta es la forma de guardar la información para que el sistema de HA lo maneje fácilmente. También podría pensarse en mantener los datos en otro servidor de datos con HA, un sistema de ficheros remoto que tenga eso, etc. De momento no se implmenta nada de eso, pero seguramente se quiera esa función en el futuro, sólo es tenerlo en mente.

Otras Especificaciones

El asunto de asingar IPs, se podría hacer integrando un IPAM, o más bien, empaquetando un IPAM junto al orquestador que trabaje para él. En futuras versiones se podrán explorar otras formas de aisgnar IPs, aparte del truco actual de codificar la IP en la MAC; como los sistemas de definir la IP en algún fichero o partición y que el arranque de linux lo use.

De cualquier manera el tema de las direcciones IP de las VM es un asunto para el futuro

Herramientas propuestas

A modo de idea, estas herramientas podrían ser útiles:

- Para el cliente: modulo cmd2 de python, para modo TUI en el cliente: Textual
- Para el servicio: framework FastAPI. Servidor web integrado con FastAPI; niceGUI
- Para cliente de servicios REST: HTTPX
- Para guardar datos locales: pydantic + TinyDB

El usuario definirá en JSON o YAML las definiciones locales o compartidas, y las subirá al orquestador con comandos o con la web. Posiblemente pueda ver o editar mediante editor integrado en el cliente TUI o GUI del servidor web.

Esto podría cambiar, es solo una referencia de posibles paquetes a usar, que parecen adecuados